# Relatório de Teste de Intrusão

Aplicação Web — OopsSec Store

Avaliação Final · Cybersecurity Offensive

**Autor:** Victor Caldas dos Santos

**Disciplina:** Exploração de Vulnerabilidade Web e API

**Curso:** Pós-graduação Cybersecurity Offensive — Pentest & Red Team

**Aplicação alvo:** OopsSec Store

**Endereço:** http://localhost:3000

**Stack:** Next.js · React · TypeScript · Prisma · SQLite

**Repositório:** github.com/kOaDT/oss-oopssec-store

**Data:** 18 de outubro de 2026

## 1. Sumário Executivo

Foi conduzido um teste de intrusão (pentest) na aplicação web **OopsSec Store**, um e-commerce construído em Next.js, com o objetivo de identificar e comprovar vulnerabilidades de segurança. Os testes foram realizados em ambiente isolado e autorizado, utilizando técnicas de interceptação de tráfego, injeção de código, manipulação de requisições e parâmetros, e análise do conteúdo entregue ao cliente.

Foram identificadas e exploradas **5 vulnerabilidades distintas**, cobrindo diferentes categorias do OWASP Top 10. Todas foram comprovadas por reprodução técnica controlada, com captura das evidências e validação pelas flags (`OSS{...}`) fornecidas pela própria aplicação. A severidade de cada item foi calculada com o **CVSS 3.1** (FIRST.org).

| # | Vulnerabilidade | OWASP / CWE | CVSS 3.1 | Severidade |
|---|---|---|---|---|
| 1 | Exposição de credencial de pagamento no bundle JavaScript (NEXT_PUBLIC_ leak) | A05:2021 · CWE-200/798 | 6.5 | **Média** |
| 2 | Insecure Direct Object Reference (IDOR) em pedidos | A01:2021 · CWE-639 | 6.5 | **Média** |
| 3 | SQL Injection no filtro de busca de pedidos | A03:2021 · CWE-89 | 7.1 | **Alta** |
| 4 | Stored XSS em avaliações de produto | A03:2021 · CWE-79 | 5.4 | **Média** |
| 5 | Path Traversal na API de documentos | A01:2021 · CWE-22 | 7.5 | **Alta** |

### Metodologia e ferramentas

Os testes seguiram uma abordagem manual de exploração. Ferramentas utilizadas: **navegador Firefox** e seu **DevTools** (abas Network, Console e Debugger), **Burp Suite** (interceptação e repetição de requisições via Repeater) e manipulação direta de parâmetros de URL e de corpo JSON. A aplicação foi executada localmente via Docker (`docker run -p 127.0.0.1:3000:3000 leogra/oss-oopssec-store`). Cada vulnerabilidade foi confirmada por reprodução técnica, e não apenas identificada teoricamente.

## 2. Vulnerabilidade 01 — Exposição de Credencial de Pagamento (NEXT_PUBLIC_ Env Leak)

**CVSS 6.5 · Média**

**Nome e classificação:** Exposição de informação sensível — vazamento de credencial via variável de ambiente `NEXT_PUBLIC_`. Severidade **Média** (CVSS 3.1: **6.5** — `AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`). OWASP A05:2021 — Security Misconfiguration. CWE-200 e CWE-798.

**Descrição:** A página de checkout (`/checkout`) lê a variável `process.env.NEXT_PUBLIC_PAYMENT_SECRET` e a anexa à requisição de pedido no header `X-Payment-Auth`. Como a variável usa o prefixo `NEXT_PUBLIC_`, o compilador do Next.js substitui a chamada pelo valor literal durante o build, embutindo a credencial no JavaScript entregue ao navegador. Qualquer visitante que carregue a página de checkout recebe o segredo. Endpoint afetado: `POST /api/orders`.

**Identificação:** Buscar o nome da variável no código-fonte não retorna resultados (o compilador o substitui pelo valor), então a descoberta foi feita observando as requisições de rede. Ferramentas: Firefox DevTools (Network e Console).

**Exploração e evidências:**

1. Autenticação com a conta de teste `alice@example.com` e adição de um produto ao carrinho.
2. Acesso a `/checkout`, aba **Network**: a requisição `POST /api/orders` carregava o header de autenticação de pagamento codificado em Base64:

```
X-Payment-Auth: T1NTe3B1YmxpY18zbnZpcjBubWVudF92NHJpNGJsM30=
```

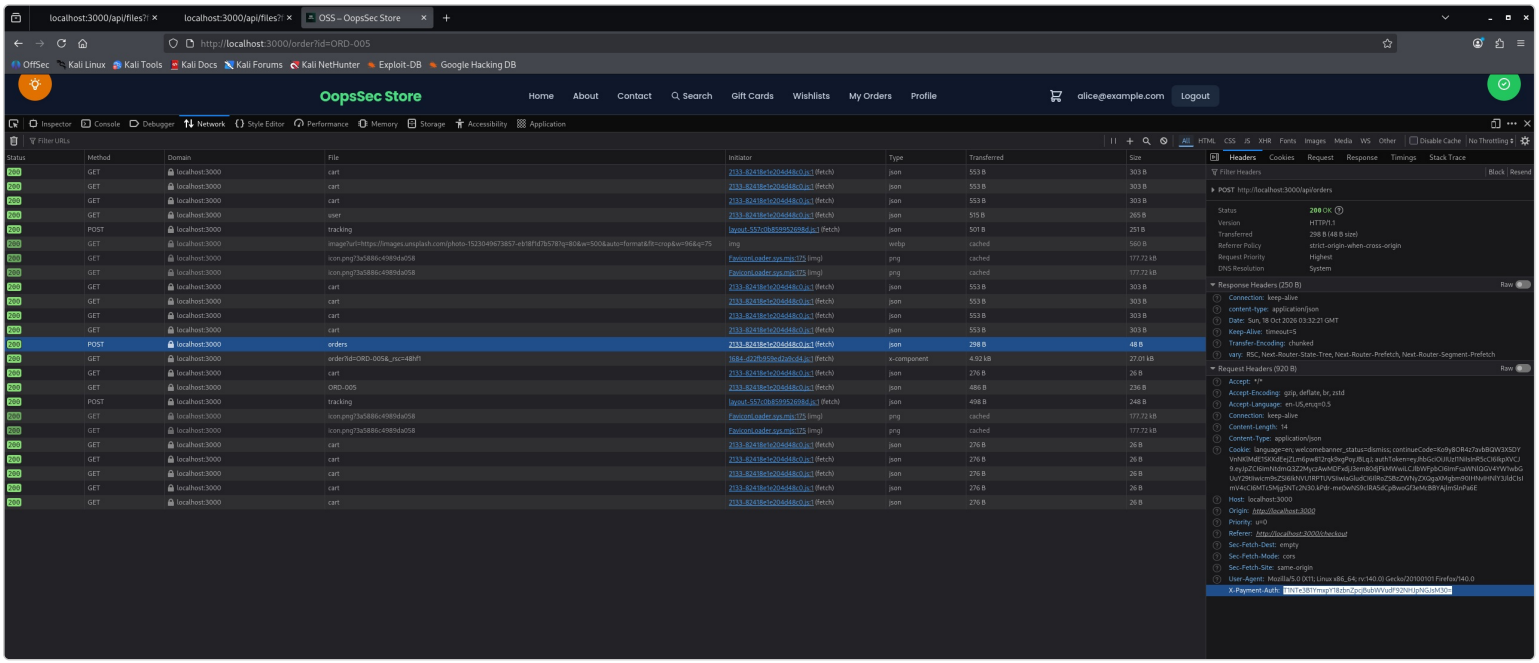


*Figura 1.1 — Aba Network exibindo o header X-Payment-Auth (Base64) na requisição POST /api/orders.*

O valor foi decodificado no Console do navegador, revelando o segredo:

```
> atob("T1NTe3B1YmxpY18zbnZpcjBubWVudF92NHJpNGJsM30=")
< "OSS{public_3nvir0nment_v4ri4bl3}"
```

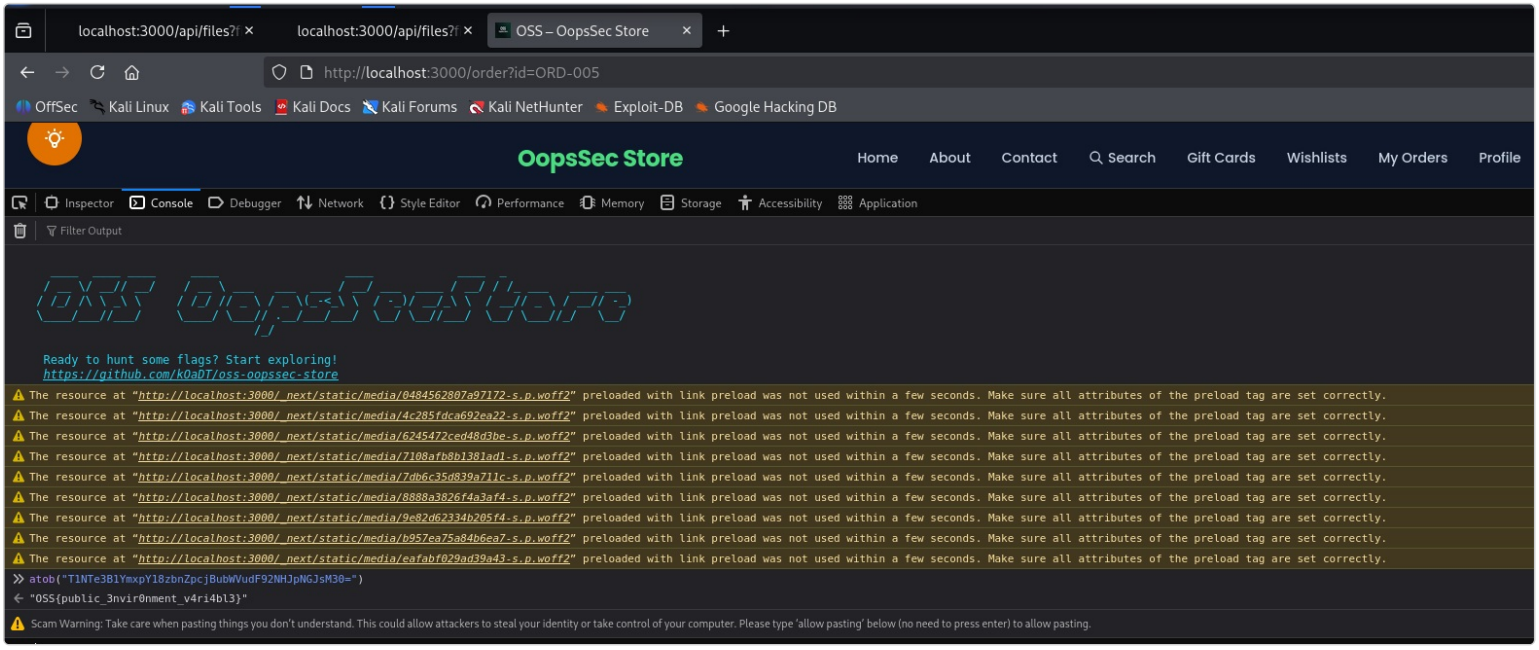


*Figura 1.2 — Console do DevTools decodificando o valor com atob(): OSS{public_3nvir0nment_v4ri4bl3}.*

**Impacto:** A credencial de pagamento fica acessível a qualquer usuário, inclusive não autenticados que acessem os arquivos estáticos. Em produção, um segredo de gateway de pagamento exposto permitiria operações fraudulentas em nome da loja. Como arquivos estáticos são cacheados por CDNs e arquivados, o segredo deve ser considerado permanentemente comprometido.

**Mitigação:** Mover a credencial para o lado servidor: remover o prefixo `NEXT_PUBLIC_` e ler o segredo apenas em route handlers / server actions. Adicionar verificação em CI (`grep -rE "NEXT_PUBLIC_.*(SECRET|KEY|TOKEN|PASSWORD)"`) e rotacionar o segredo exposto.

## 3. Vulnerabilidade 02 — Insecure Direct Object Reference (IDOR) em Pedidos CVSS 6.5 · Média

**Nome e classificação:** Falha de controle de acesso — IDOR. Severidade **Média** (CVSS 3.1: **6.5** — `AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`). OWASP A01:2021 — Broken Access Control. CWE-639.

**Descrição:** A página de detalhes de pedido (`/order?id=ORD-XXX`) identifica o pedido por um parâmetro `id` sequencial e previsível. O servidor retorna os dados sem verificar se o pedido pertence ao usuário autenticado, permitindo que qualquer usuário logado leia pedidos de terceiros apenas alterando o identificador na URL.

**Identificação:** Observando a URL de um pedido próprio (`ORD-004`), o formato sequencial sugeriu enumeração. Ferramenta: navegador (manipulação direta da URL).

**Exploração e evidências:**

1. Autenticado como `alice@example.com`, acesso ao próprio pedido em `/order?id=ORD-004`.
2. Alteração do parâmetro para `/order?id=ORD-002`.
3. A aplicação retornou os dados completos de um pedido de outro cliente (**Bob** — `bob@example.com`, endereço em Berlim), confirmando o acesso não autorizado, validado pela flag:

```
OSS{1ns3cur3_d1r3ct_0bj3ct_r3f3r3nc3}
```

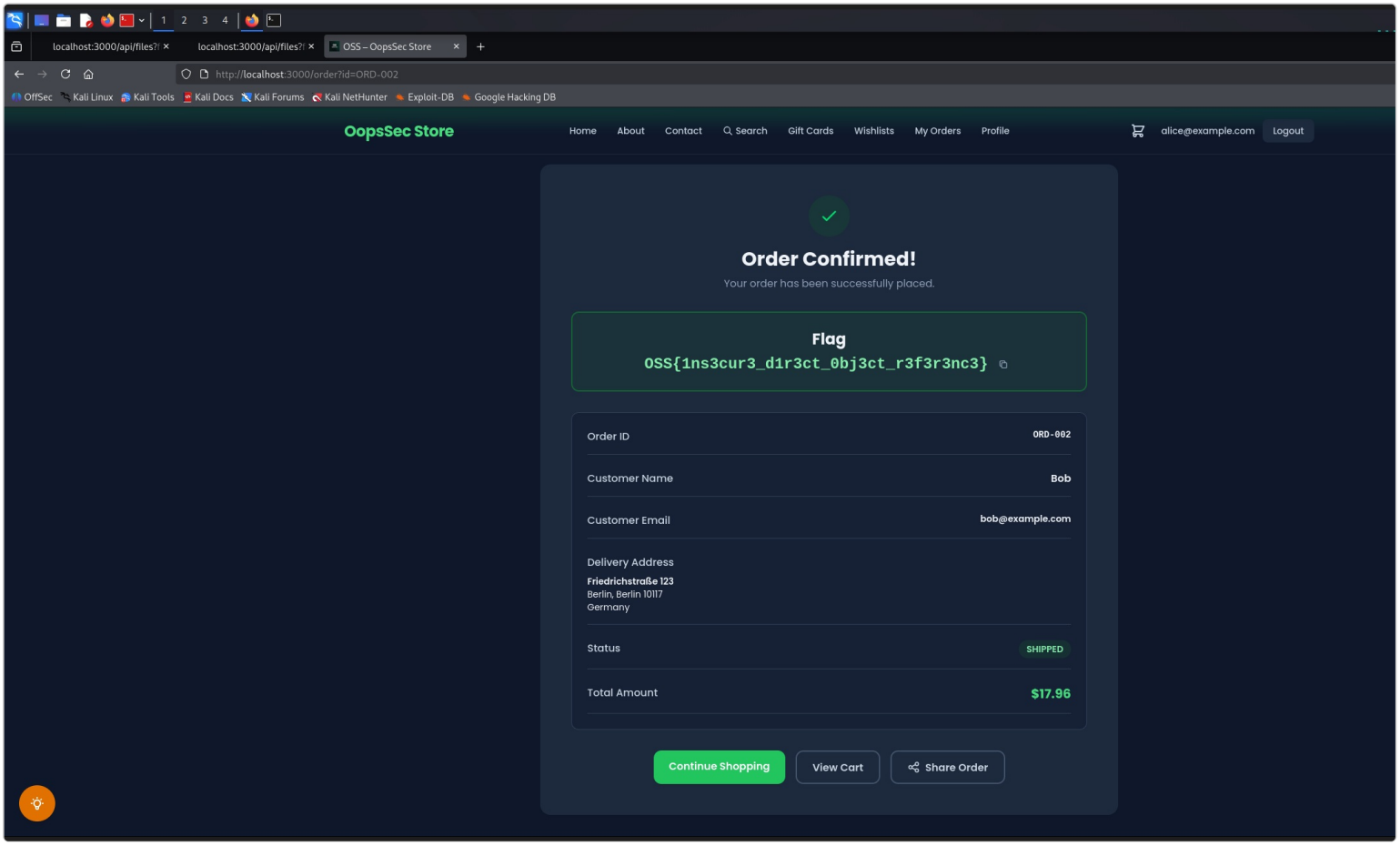



*Figura 2.1 — Sessão da Alice (alice@example.com) visualizando o pedido ORD-002 do cliente Bob, com a URL e a flag.*

**Impacto:** Qualquer usuário autenticado pode enumerar e ler todos os pedidos da plataforma, expondo dados pessoais de outros clientes (nome, e-mail, endereço), histórico de compras e valores — vazamento massivo de dados pessoais (violação da LGPD) e quebra da segregação entre contas.

**Mitigação:** Verificar a propriedade no servidor (comparar o `userId` da sessão com o dono do pedido) antes de retornar os dados. Como defesa em profundidade, usar identificadores não previsíveis (UUIDs).

## 4. Vulnerabilidade 03 — SQL Injection no Filtro de Busca de Pedidos CVSS 7.1 · Alta

**Nome e classificação:** Injeção de SQL (SQL Injection). Severidade **Alta** (CVSS 3.1: **7.1** — `AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:N`). OWASP A03:2021 — Injection. CWE-89.

**Descrição:** O endpoint `POST /api/orders/search` recebe o parâmetro `status` e o concatena diretamente numa query SQL executada via `prisma.$queryRawUnsafe()`, sem parametrização, permitindo injeção de SQL arbitrário.

**Identificação:** Capturada a requisição no Burp Suite; enviado o parâmetro `status` com uma aspa simples, provocando erro. Ferramenta: Burp Suite (Repeater).

**Exploração e evidências:**

1. Payload com aspa solta retornou **HTTP 500** com erro do Prisma vazando a query interna, confirmando a injeção:

```
{"status":"DELIVERED'"}
```

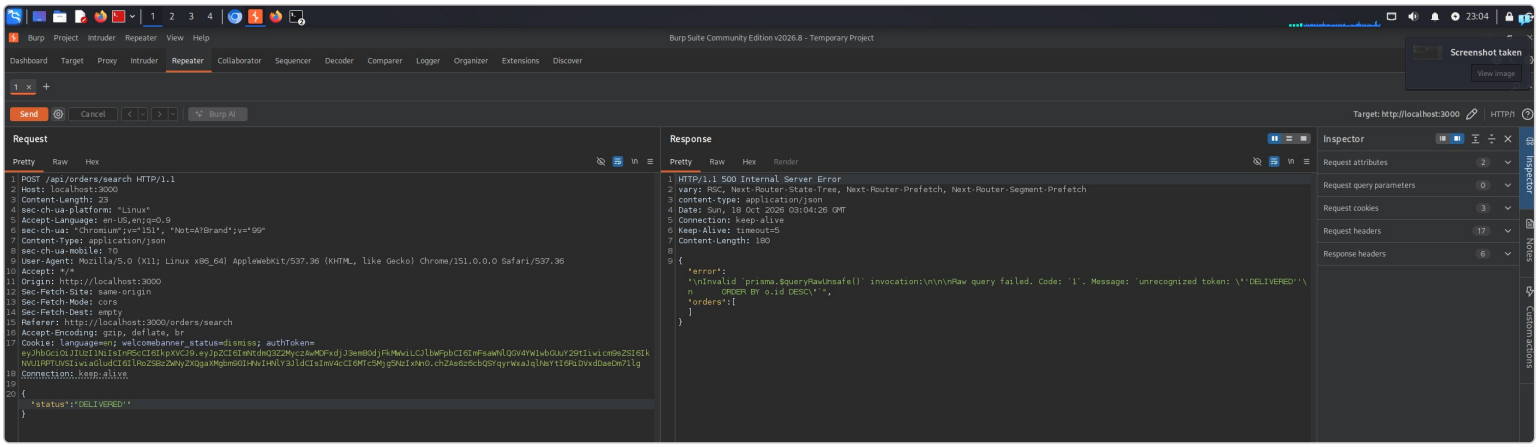


*Figura 3.1 — Erro HTTP 500 do Prisma (`$queryRawUnsafe`) revelando a estrutura da query.*

2. UNION SELECT de 9 colunas sobre a tabela `users` retornou e-mails, hashes MD5 e papéis de todos os usuários, incluindo a conta **admin@oss.com** (role ADMIN):

```
{"status":"DELIVERED' UNION SELECT id, email, password, role, addressId, id, email, password, role FROM users --"}
```

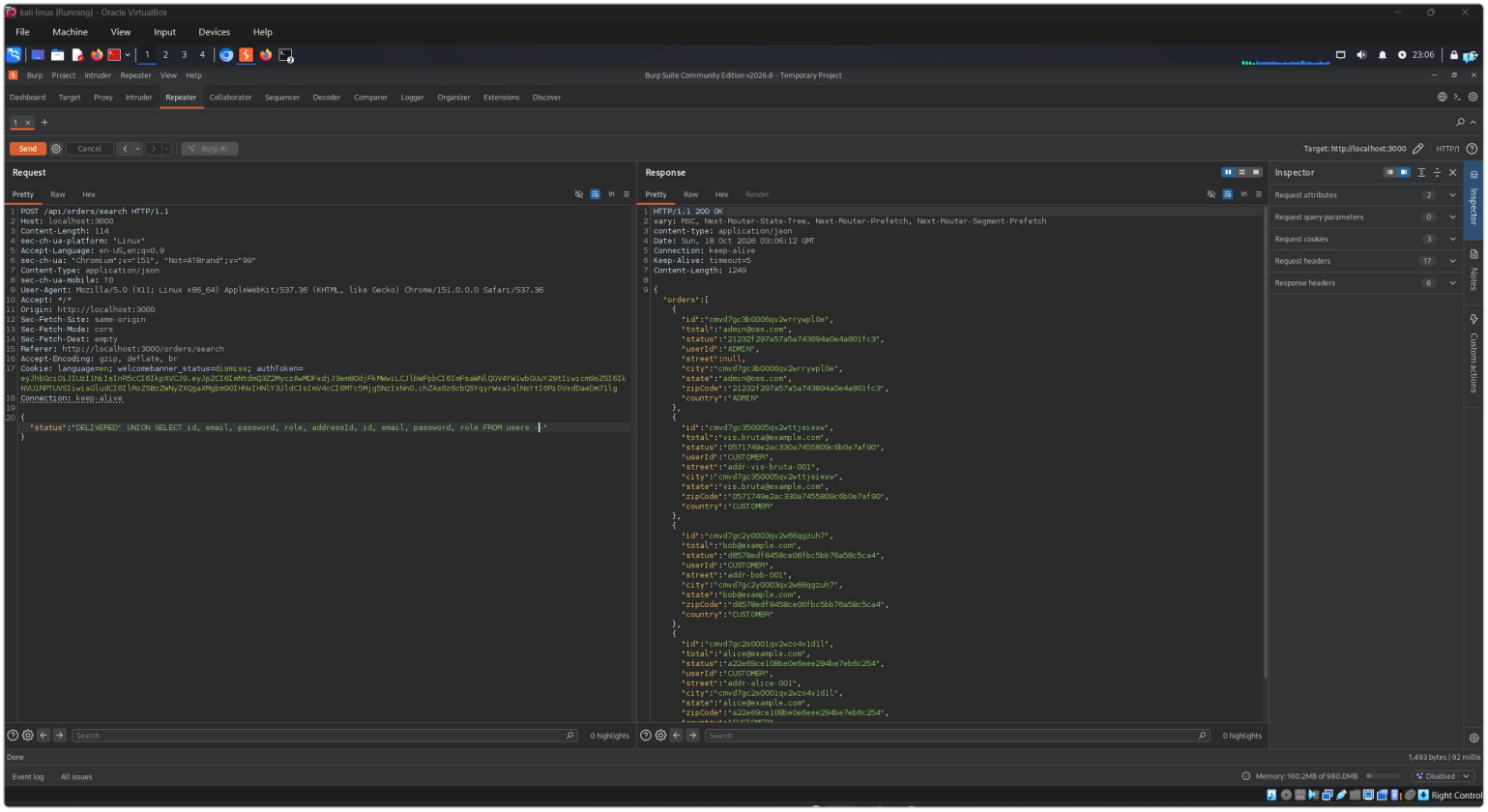


*Figura 3.2 — Dump da tabela users: e-mails, hashes MD5 e papéis (incluindo o administrador).*

3. Validação via flag extraída da tabela `internal_secrets`:

```
OSS{sql_1nj3ct10n_vuln3r4b1l1ty}
```

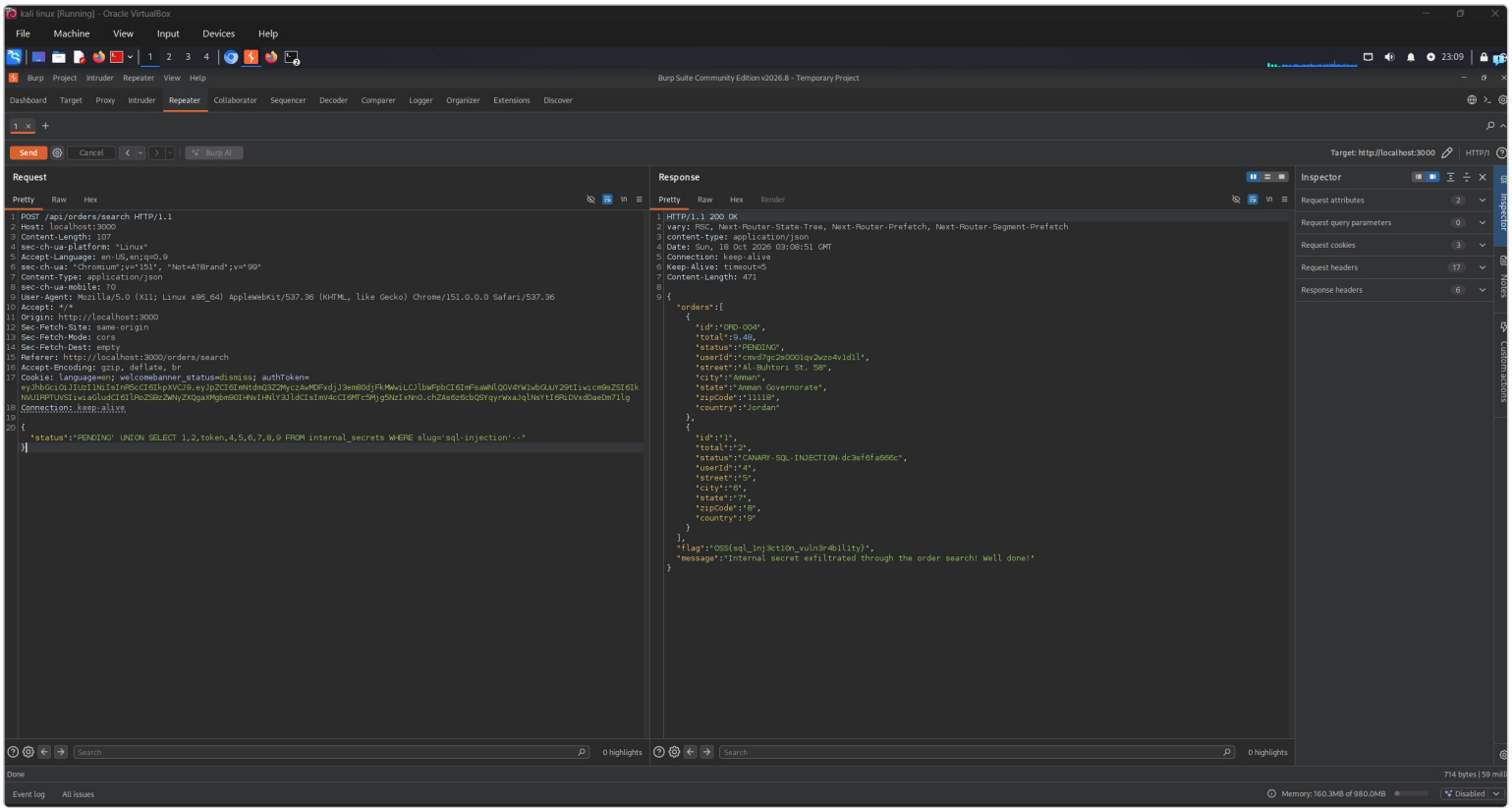



*Figura 3.3 — Segredo interno exfiltrado e flag liberada pela aplicação.*

**Impacto:** Comprometimento total da confidencialidade do banco — qualquer usuário autenticado extrai todas as credenciais. Agravante: as senhas usam **MD5 sem salt**, trivialmente quebrável (o hash do admin corresponde à senha "admin"), levando a tomada de conta administrativa. A injeção também permite alterar/excluir dados.

**Mitigação:** Substituir a concatenação por **queries parametrizadas** (ex.: `prisma.findMany` com `where`). Validar o `status` contra uma allowlist. Tratar erros sem expor detalhes internos. Migrar o hash de senhas para bcrypt/argon2.

## 5. Vulnerabilidade 04 — Stored XSS em Avaliações de Produto CVSS 5.4 · Média

**Nome e classificação:** Cross-Site Scripting armazenado (Stored XSS). Severidade **Média** (CVSS 3.1: **5.4** — `AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N`). OWASP A03:2021 — Injection. CWE-79.

**Descrição:** O campo de avaliação (review) de produtos aceita e armazena texto sem sanitização, e a aplicação o injeta como HTML cru no DOM, executando scripts embutidos. Por ser persistido, o payload dispara no navegador de todos os usuários que abrirem a página. Endpoint: `POST /api/products/[id]/reviews`.

**Identificação:** Submetida uma avaliação com uma tag `<script>` em uma página de produto. Ferramenta: navegador.

**Exploração e evidências:**

1. Payload de confirmação disparou um popup ao carregar os reviews:

```
<script>alert("XSS")</script>
```

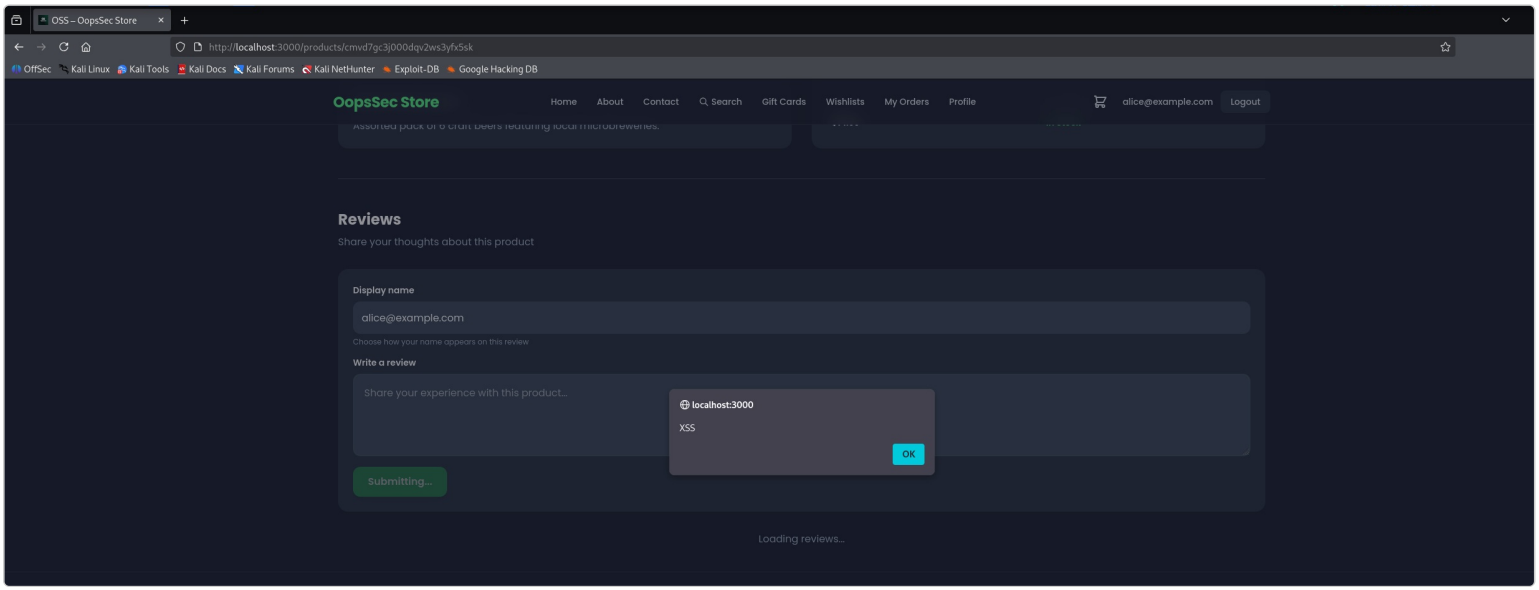



*Figura 4.1 — Execução de JavaScript arbitrário (popup "XSS") a partir da avaliação armazenada.*

2. Para demonstrar impacto real, o payload leu um arquivo do servidor e exibiu seu conteúdo, provando exfiltração no contexto da vítima:

```
<script>fetch("/xss-flag.txt").then(r=>r.text()).then(f=>alert("Flag: "+f))</script>
```

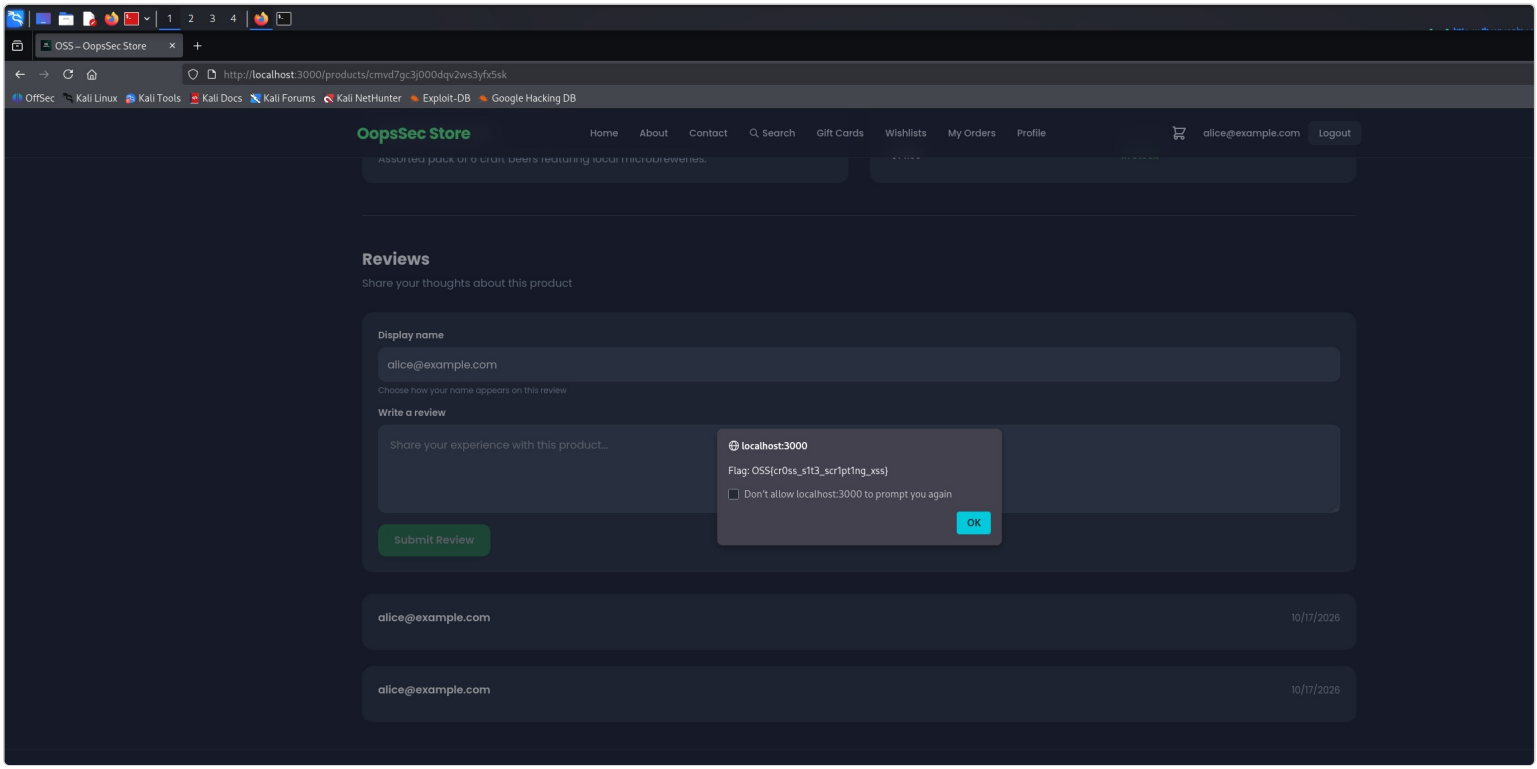



*Figura 4.2 — Exfiltração de arquivo via XSS (flag OSS{cr0ss_s1t3_scr1pt1ng_xss}); abaixo, as avaliações maliciosas persistidas.*

**Impacto:** Por ser armazenado, atinge qualquer visitante da página sem interação além de abri-la. Permite roubo de cookies/tokens de sessão, ações em nome da vítima, redirecionamento malicioso e exfiltração de dados — inclusive comprometer administradores.

**Mitigação:** Escapar/sanitizar toda saída (output encoding contextual); nunca injetar conteúdo como HTML cru (evitar `dangerouslySetInnerHTML`); aplicar uma Content Security Policy (CSP) restritiva.

## 6. Vulnerabilidade 05 — Path Traversal na API de Documentos CVSS 7.5 · Alta

**Nome e classificação:** Travessia de diretório (Path Traversal). Severidade **Alta** (CVSS 3.1: **7.5** — `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`). OWASP A01:2021 — Broken Access Control. CWE-22.

**Descrição:** O endpoint `GET /api/files` recebe o parâmetro `file` e o concatena ao diretório `documents/` sem validação ou normalização. Sequências `../` permitem escapar do diretório e ler arquivos arbitrários do servidor.

**Identificação:** Testado o endpoint com um arquivo legítimo e, em seguida, com sequências de travessia. Ferramenta: navegador.

**Exploração e evidências:**

1. Comportamento normal com um arquivo válido dentro do diretório:

```
http://localhost:3000/api/files?file=readme.txt
```

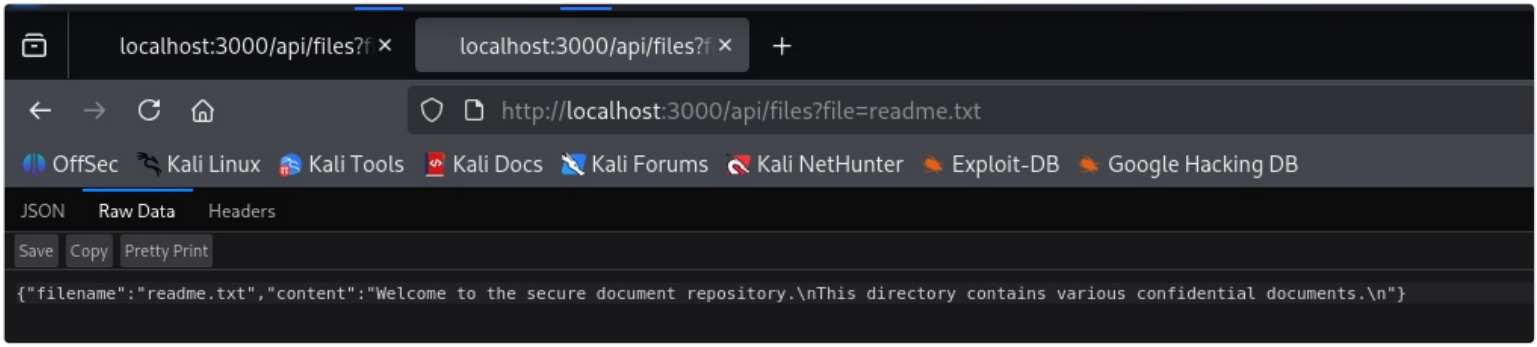



*Figura 5.1 — Leitura legítima de um documento dentro do diretório `documents/`.*

2. Com `../`, a leitura escapou do diretório e acessou um arquivo da raiz do projeto:

```
http://localhost:3000/api/files?file=../flag.txt
```

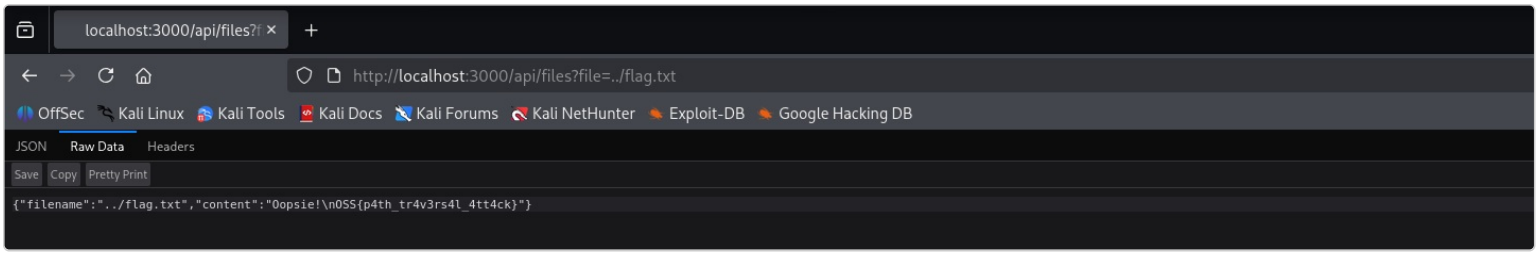



*Figura 5.2 — Travessia de diretório lendo `../flag.txt` fora do diretório pretendido (flag OSS{p4th_tr4v3rs4l_4tt4ck}).*

**Impacto:** Leitura arbitrária de arquivos do servidor sem necessidade de autenticação. Além do arquivo de flag, alvos como `../.env` (segredos da aplicação, chaves de API, credenciais de banco) e `../../../../etc/passwd` (arquivos do sistema) tornam-se acessíveis, podendo encadear para comprometimento mais profundo.

**Mitigação:** Normalizar o caminho (`path.normalize`) e validar que o caminho resolvido permanece dentro do diretório permitido; rejeitar entradas contendo `../`; usar uma allowlist de arquivos servíveis; executar o processo com privilégios mínimos.

## 7. Conclusão

A aplicação OopsSec Store apresentou falhas de segurança significativas em múltiplas categorias do OWASP Top 10, abrangendo controle de acesso quebrado (IDOR e Path Traversal), injeção (SQL Injection e Stored XSS) e configuração insegura (vazamento de credencial). As duas vulnerabilidades de severidade **Alta** — SQL Injection e Path Traversal — são as mais críticas: a primeira expõe todas as credenciais da base (agravada pelo uso de MD5 sem salt), e a segunda permite leitura arbitrária de arquivos sem autenticação. A causa raiz recorrente é a **confiança em entradas controladas pelo cliente sem validação no servidor**. Recomenda-se, de forma prioritária, a adoção de queries parametrizadas, sanitização e escape de saída, verificação de autorização por objeto e normalização/validação de caminhos de arquivo, além da migração do algoritmo de hash de senhas.

— Fim do relatório · 5 vulnerabilidades comprovadas —